

CodePilot-AI

PROJECT REPORT

on

CodePilot-AI

AI-Powered Code Review & Security Analysis Platform

ABSTRACT

Code review is one of the most effective ways to find defects, security flaws and maintainability problems before software reaches production, but it is time-consuming and depends on the availability and experience of reviewers. Students and individual developers, in particular, often have no one to review their code.

CodePilot AI is a full-stack web application that provides automated, AI-assisted code review. A registered user pastes source code, selects a programming language and receives a structured review containing a summary, a code-quality score from 0 to 100, a list of issues with category, severity and line number, improvement suggestions, and a rewritten (improved) version of the code. The user can then ask follow-up questions about the code in an AI chat, download the review as a PDF report, and revisit earlier reviews from a history page. A dashboard summarises the user's activity.

The system uses a React (Vite, Tailwind CSS) front end deployed on Vercel, a FastAPI (Python) back end deployed on Render, MongoDB Atlas for persistence, and the Groq API (model openai/gpt-oss-120b) for analysis. Authentication is implemented with bcrypt-hashed passwords and JSON Web Tokens. The AI response is requested in JSON mode and validated and normalised on the server before it is stored and returned, so that the front end always receives a predictable structure.

TABLE OF CONTENTS

Chapter 1: Introduction	6
1.1 Background	6
1.2 Problem Statement	6
1.3 Objectives	6
1.4 Scope of the Project	6
1.5 Organisation of the Report	6
Chapter 2: Existing and Proposed System	8
2.1 Existing System	8
2.2 Proposed System	8
Chapter 3: System Analysis	9
3.1 Functional Requirements	9
3.2 Non-Functional Requirements	9
3.3 Feasibility Study	9
3.4 Software and Hardware Requirements	10
Chapter 4: System Design	11
4.1 Architecture	11
4.2 Application Workflow	11
4.3 Module Description	11
4.4 Database Design	12
4.5 API Design	13
4.6 Security Design	13
Chapter 5: Technology Stack	14
Chapter 6: Implementation	15
6.1 Project Structure	15
6.2 Back-end Implementation	15
6.3 Front-end Implementation	16
6.4 Sample Request and Response	17
Chapter 7: Testing	18
7.1 Testing Approach	18
7.2 Test Cases	18
Chapter 8: Deployment and Configuration	19
8.1 Deployment	19
8.2 Environment Variables	19
8.3 Running Locally	19
Chapter 9: Limitations	20

Chapter 10: Future Enhancements	21
Chapter 11: Conclusion.....	22
References.....	23
Appendix A: Screenshots.....	24

Chapter 1: Introduction

1.1 Background

Software quality depends heavily on how well code is reviewed before it is merged and released. A review can reveal logic errors, insecure patterns such as SQL injection or hard-coded secrets, inefficient algorithms and violations of coding conventions. In practice, however, reviews are limited by time, reviewer experience and availability.

Large language models (LLMs) have become capable of reading source code, explaining it and proposing fixes. This makes it feasible to build a tool that gives every developer an on-demand first-pass reviewer. CodePilot AI is built on this idea.

1.2 Problem Statement

Developers, and especially students and beginners, frequently submit code without any review. Manual review by a senior engineer is not always available, static-analysis tools are often language-specific and hard to configure, and their output is difficult for beginners to interpret. There is a need for an easy-to-use web application that analyses code, explains the problems in plain language, provides a corrected version, and allows follow-up questions.

1.3 Objectives

- To build a secure web application with user registration and login.
- To analyse submitted source code using an LLM and detect bugs, security vulnerabilities, performance problems and poor practices.
- To present results in a structured form: summary, quality score, categorised issues with severity and line numbers, suggestions and improved code.
- To allow users to ask follow-up questions about the reviewed code through an AI chat.
- To store every review so that users can browse their history and view statistics on a dashboard.
- To let users download a review as a PDF report.
- To deploy the system on free cloud platforms so that it is publicly accessible.

1.4 Scope of the Project

The application targets individual developers and students. In the current version the review page supports Python, JavaScript and TypeScript. Code is entered through an in-browser editor; reviews are stored per user. Repository integration, pull-request review and team features are outside the present scope and are listed under future enhancements.

1.5 Organisation of the Report

Chapter 2 reviews the existing approaches and the proposed system. Chapter 3 presents the system analysis and requirements. Chapter 4 describes the system design. Chapter 5 covers the technology stack. Chapter 6 explains the implementation. Chapter 7 describes testing. Chapter 8 covers deployment and configuration. Chapter 9 lists limitations and Chapter 10 future enhancements, followed by the conclusion, references and appendices.

Chapter 2: Existing and Proposed System

2.1 Existing System

Code is currently reviewed in one of three ways:

- **Manual peer review:** accurate and contextual, but slow and dependent on another person's time.
- **Static analysis tools and linters:** fast and automated, but usually limited to one language or ruleset, require setup, and produce terse messages that beginners find hard to act on.
- **General-purpose AI chatbots:** flexible, but the user must write the prompt, the output is unstructured, nothing is stored, and there is no scoring or history.

Drawbacks of the existing approaches

- No instant, always-available reviewer for individual developers.
- Output is not structured, so results cannot be compared over time.
- No built-in record of previous reviews or progress.
- Fixes are described but a corrected version of the code is not always provided.

2.2 Proposed System

CodePilot AI wraps an LLM in a purpose-built workflow. The server instructs the model, through a carefully designed prompt, to act as a senior software engineer and security reviewer and to return strict JSON. The response is validated, normalised and stored. The user receives a clear, consistent review each time, can discuss it with the AI, and can track reviews over time.

Advantages of the proposed system

- Instant review of pasted code with no tooling to install.
- Consistent structured output: summary, score, issues, suggestions and improved code.
- Security-focused analysis along with quality and performance.
- AI chat that has the original and improved code as context.
- Review history, dashboard statistics and downloadable PDF reports.
- Secure per-user data with JWT-protected endpoints.

Chapter 3: System Analysis

3.1 Functional Requirements

ID	Requirement
FR1	A visitor can register with name, email and password; duplicate emails are rejected.
FR2	A registered user can log in and receives a JWT access token.
FR3	Only authenticated users can access the dashboard, review, history and profile pages.
FR4	The user can select a language, enter code and request a review.
FR5	The system returns a summary, score (0–100), issues, suggestions and improved code.
FR6	The user can ask follow-up questions about the reviewed code through an AI chat.
FR7	Every review is saved against the user and listed in the history page.
FR8	The user can open a single past review in detail.
FR9	The dashboard shows total reviews, average score, issue count and number of languages used.
FR10	The user can download a review report as a PDF.
FR11	The profile page shows account information and statistics.

3.2 Non-Functional Requirements

- **Security:** passwords must be stored hashed; protected endpoints must verify a valid token; secrets must not be committed to source control.
- **Usability:** a responsive interface with clear loading states and error handling.
- **Performance:** the interface must remain responsive while the AI analysis runs; history and recent-review queries exclude large code fields.
- **Reliability:** if the AI service fails or returns invalid JSON, the system returns a safe fallback response instead of crashing.
- **Maintainability:** separation into routes, services, models, utilities and database modules on the back end and pages, components and API layer on the front end.
- **Portability:** configuration through environment variables so the same code runs locally and in the cloud.

3.3 Feasibility Study

Technical feasibility: all technologies used (React, FastAPI, MongoDB, Groq API) are mature, well documented and free to use at the scale required.

Economic feasibility: the project is built entirely with free tiers of Vercel, Render, MongoDB Atlas and the Groq API, so there is no infrastructure cost.

Operational feasibility: the application runs in a web browser with no installation, and the workflow (paste code, click review) is simple for the target users.

3.4 Software and Hardware Requirements

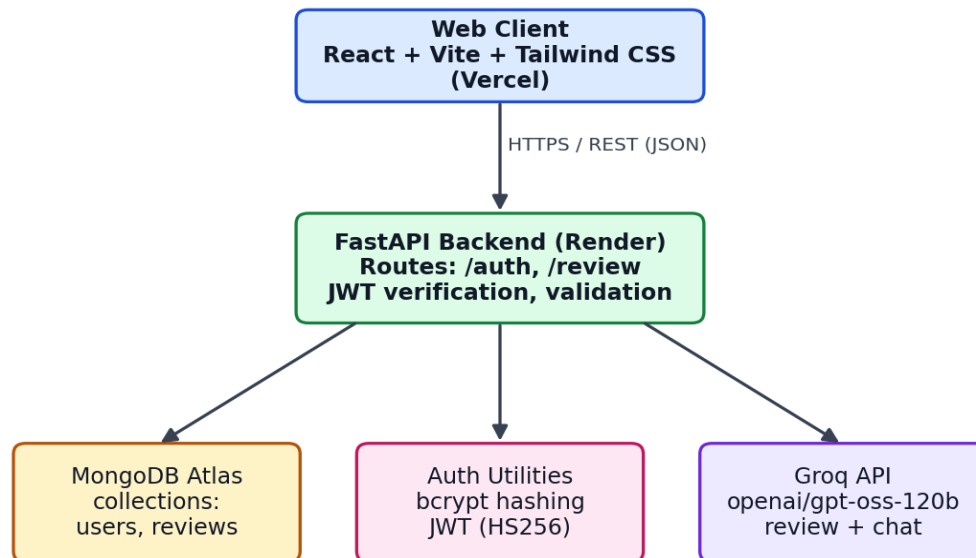
Item	Requirement
Operating system	Windows / Linux / macOS
Back end	Python 3, FastAPI, Uvicorn, PyMongo, Groq SDK, python-jose, passlib with bcrypt
Front end	Node.js and npm; React, Vite, Tailwind CSS, React Router, Axios
Database	MongoDB Atlas (cloud)
AI service	Groq API key
Browser	Any modern browser (Chrome, Edge, Firefox, Safari)
Hardware (development)	Any computer with at least 4 GB RAM and an internet connection

Chapter 4: System Design

4.1 Architecture

CodePilot AI follows a three-tier client-server architecture. The React client runs in the browser and communicates with the FastAPI server over HTTPS using JSON. The server authenticates requests, calls the Groq API for AI analysis and stores data in MongoDB Atlas.

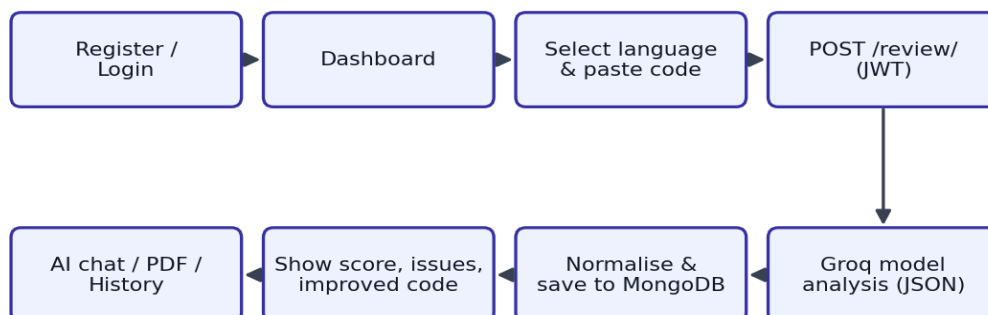
Figure 4.1: System Architecture of CodePilot AI



4.2 Application Workflow

A user registers or logs in, lands on the dashboard, opens the review page, selects a language and submits code. The server calls the model, normalises the result, stores it and returns it. The user can then chat with the AI, download a PDF or revisit the review later from history.

Figure 4.2: Application Workflow



4.3 Module Description

Module	Description
Authentication	Registration, login and current-user endpoints. Passwords are hashed with bcrypt; a signed JWT is issued on login and verified on every protected request.
Code Review	Accepts language and code, builds the analysis prompt, calls the Groq model in JSON mode, normalises the response and stores it.
AI Chat	Answers user questions using the original code, the improved code and the language as context.
Review History	Lists a user's previous reviews (without the large code fields) newest first, and returns a single review in full.
Dashboard	Shows statistics (total reviews, average score, total issues, languages used), quick actions and recent reviews.
Profile	Displays account details from the current-user endpoint together with review statistics.
Report Generation	Builds a PDF of the review (summary, score, issues) in the browser using jsPDF.

4.4 Database Design

MongoDB is a document database, so the data is stored as two collections with flexible schemas. Reviews are linked to users through the user's email address.

Collection: users

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier
name	String	Display name (trimmed)
email	String	Lower-cased email; used as the login identifier
password	String	bcrypt hash of the password (the plain password is never stored)

Collection: reviews

Field	Type	Description
_id	ObjectId	Auto-generated unique identifier
user_email	String	Email of the user who owns the review
language	String	Programming language selected
code	String	Original source code submitted
summary	String	Short AI summary of the review
score	Integer	Code-quality score between 0 and 100
issues	Array	Objects with title, category, severity, line, description
suggestions	Array	List of improvement suggestions
improved_code	String	Rewritten code produced by the AI
created_at	DateTime	UTC timestamp of the review

4.5 API Design

Method	Endpoint	Auth	Purpose
GET	/	No	Health check; returns status message and database name
POST	/auth/register	No	Create a new account
POST	/auth/login	No	Verify credentials and return a JWT
GET	/auth/me	Yes	Return the current user (password excluded)
POST	/review/	Yes	Analyse code, save and return the review
POST	/review/chat	Yes	Answer a follow-up question about the code
GET	/review/history	Yes	List the user's reviews (newest first)
GET	/review/stats	Yes	Totals and averages for the dashboard
GET	/review/recent	Yes	Three most recent reviews

Method	Endpoint	Auth	Purpose
GET	/review/{id}	Yes	A single review owned by the user

4.6 Security Design

- Passwords are hashed with bcrypt through passlib before storage.
- A JWT signed with HS256 and a server-side secret is issued at login with a 7-day expiry.
- Protected endpoints use an HTTP Bearer dependency that decodes and validates the token.
- Review lookups are filtered by the authenticated user's email so one user cannot read another's reviews.
- CORS is restricted to the local development origins and the deployed front-end URL.
- API keys, database URL and the JWT secret are read from environment variables and the .env file is excluded by .gitignore.
- On the client, an Axios interceptor attaches the token to each request and redirects to the login page on a 401 response.

Chapter 5: Technology Stack

Layer	Technology	Reason for choice
Front end	React 19, Vite, Tailwind CSS 4, React Router 7	Component-based UI, fast development server and utility-first styling
UI libraries	lucide-react, react-icons, framer-motion, MUI, recharts	Icons, animation and UI building blocks
Code display	react-syntax-highlighter	Syntax-highlighted display of code
PDF	jsPDF	Generates the PDF report in the browser without a server
HTTP client	Axios	Interceptors for token handling and global error handling
Back end	Python, FastAPI, Uvicorn, Pydantic	Fast, typed REST API with automatic request validation and docs
Database	MongoDB Atlas with PyMongo	Flexible document model suits nested review data
AI	Groq API, model openai/gpt-oss-120b	Low-latency inference and JSON-mode output
Authentication	python-jose (JWT), passlib + bcrypt	Stateless tokens and strong password hashing
Deployment	Vercel (front end), Render (back end), GitHub	Free hosting with Git-based deployment

Chapter 6: Implementation

6.1 Project Structure

CodePilot-AI/

```
|— backend/
| |— app/
| | |— database/mongodb.py    # MongoDB client
| | |— models/                # Pydantic request models
| | |— routes/auth.py         # register, login, me
| | |— routes/review.py       # review, chat, history, stats
| | |— services/ai_reviewer.py # code analysis via Groq
| | |— services/chat_ai.py    # follow-up chat via Groq
| | |— utils/security.py      # hashing, JWT creation
| | |— utils/auth.py          # JWT dependency
| | |— config.py, main.py
| |— requirements.txt, .env.example
|— client/
| |— src/api/axios.js         # API client + interceptors
| |— src/pages/               # Home, Login, Register, Dashboard,
| |                             # Review, History, ReviewDetails, Profile
| |— src/components/          # auth, dashboard, review, home, layout
| |— src/routes/              # AppRoutes, ProtectedRoute
|— vercel.json, package.json
```

6.2 Back-end Implementation

6.2.1 Application setup

main.py creates the FastAPI application, configures CORS with the allowed origins and registers the authentication and review routers. config.py loads settings (MongoDB URL, database name, Groq key) through pydantic-settings, and mongodb.py creates a single shared MongoClient.

6.2.2 Authentication

On registration the name and email are trimmed and the email is lower-cased. The server checks for an existing account, hashes the password with bcrypt and inserts the user. Database errors

are converted to a 503 response. On login the stored hash is verified and a JWT containing the user's email and an expiry time is returned. The `get_current_user` dependency decodes the bearer token and rejects invalid or expired tokens with a 401 response.

6.2.3 AI code review service

`analyze_code()` in `ai_reviewer.py` builds a prompt that tells the model to act as a senior software engineer, security engineer and code reviewer and to review only the supplied code. The prompt fixes the JSON schema (summary, score, issues, suggestions, improved_code), gives a scoring guide (for example, 90–100 excellent, 0–39 very poor), and lists 18 rules such as “return only JSON”, “never invent vulnerabilities” and “improved code must not simply copy the original when issues exist”. The request is sent to the Groq chat-completions API with model `openai/gpt-oss-120b`, temperature 0.2 and `response_format` set to `json_object`.

Because LLM output cannot be trusted to be well formed, the response is post-processed:

- If the API call fails or the JSON cannot be parsed, a safe fallback review (score 50, a “System” issue and the original code) is returned.
- Missing fields receive defaults; the score is converted to an integer and clamped to the range 0–100.
- `improved_code` is cleaned: escaped newlines and quotes are restored, markdown fences are removed, and if the field is empty, too short, or itself looks like JSON, the original code is used instead.
- Each issue is normalised to title, category, severity, line and description; suggestions given as objects are reduced to text.

6.2.4 Review, history and statistics endpoints

`POST /review/` calls the analyser, builds a review document with the user's email, the language, the original code, the AI result and a UTC timestamp, inserts it into the reviews collection and returns the result. The history and recent endpoints exclude the code and `improved_code` fields to keep responses small. The stats endpoint computes the total number of reviews, the rounded average score, the total number of issues and the number of distinct languages for the current user. The single-review endpoint filters on both the review id and the user's email, returning 404 if the review does not belong to the user.

6.2.5 AI chat

`chat_with_code()` sends the language, original code, improved code and the question to the same model (temperature 0.3) with instructions to answer only the question, be accurate and concise, and give code where useful. If the service fails, a friendly fallback message is returned.

6.3 Front-end Implementation

6.3.1 Routing and protection

AppRoutes defines the public routes (home, login, register) and wraps the dashboard, review, history and profile pages in ProtectedRoute, which redirects to /login when no token is present in local storage. Individual review pages are served by ReviewDetails at /review/:id and /history/:id, and unknown paths show a NotFound page. vercel.json rewrites all paths to index.html so client-side routing works after deployment.

6.3.2 API layer

A single Axios instance uses the VITE_API_URL environment variable as its base URL. A request interceptor adds the Authorization header and a response interceptor removes the token and redirects to the login page on a 401 response.

6.3.3 Pages

Page	Function
Home	Landing page with animated hero section, feature overview and calls to action.
Login / Register	Authentication forms with validation and error messages; a successful login stores the token.
Dashboard	Welcome banner, statistics cards, quick actions, recent reviews and chart panels.
Review	Language selector (Python, JavaScript, TypeScript), starter snippets, code editor with copy, clear and fullscreen options, animated analysis phases while waiting, and the result view.
Review result	Score and summary, issue cards with severity, suggestions, improved code with syntax highlighting, AI chat and PDF download.
History	Searchable list of earlier reviews as cards.
Review details	Opens one saved review in full.
Profile	Account details and statistics.

6.3.4 PDF report

DownloadReport uses jsPDF to create an A4 PDF that contains the title, date, language, the overall review, the quality score and the list of issues with severity and line number.

6.4 Sample Request and Response

Request body of POST /review/:

```
{  
  "language": "python",  
  "code": "query = \"SELECT * FROM users WHERE id = '\" + user_id + '\"\"",  
}
```

Structure of the response (values illustrative):

```
{  
  "summary": "The query is built by string concatenation ...",  
  "score": 35,  
  "issues": [  
    { "title": "SQL injection", "category": "Security",  
      "severity": "High", "line": 1,  
      "description": "User input is concatenated into the query." }  
  ],  
  "suggestions": ["Use parameterised queries."],  
  "improved_code": "query = \"SELECT * FROM users WHERE id = %s\\n...\"",  
}
```

Chapter 7: Testing

7.1 Testing Approach

The application was tested functionally through the user interface and the API. The test cases below describe the scenario and the expected behaviour for each feature. The “Actual result / Status” column should be completed with the observed outcome (and screenshots) when the tests are executed.

7.2 Test Cases

ID	Scenario	Expected result	Actual / Status
T1	Register with valid name, email and password	Account created; success message	Pass
T2	Register with an email that already exists	400 error: account already exists	Pass
T3	Register with an invalid email format	Request rejected by validation	Pass
T4	Log in with correct credentials	JWT returned; user redirected to dashboard	Pass
T5	Log in with wrong password	401 Invalid email or password	Pass
T6	Open /dashboard without logging in	Redirected to /login	Pass
T7	Call /review/ with no or an invalid token	401 Invalid or expired token	Pass
T8	Submit Python code containing a SQL-injection pattern	Security issue reported; improved code uses parameterised query	Pass
T9	Submit clean, well-written code	High score; few or no issues	Pass
T10	Submit empty code	No request is sent (button does nothing)	Pass
T11	Groq API unavailable or key invalid	Fallback review returned; application does not crash	Pass
T12	Ask a follow-up question in the AI chat	Relevant answer using the code context	Pass

ID	Scenario	Expected result	Actual / Status
T13	Open History after several reviews	Reviews listed newest first	Pass
T14	Open a review by id belonging to another user	404 Review not found	Pass
T15	Download the PDF report	PDF contains summary, score and issues	Pass
T16	Open a non-existent URL	Not-found page is shown	Pass

Chapter 8: Deployment and Configuration

8.1 Deployment

Component	Platform	URL
Front end	Vercel	https://codepilot-ai-nine.vercel.app/
Back end	Render	https://codepilot-ai-backend-d7wx.onrender.com/
Database	MongoDB Atlas	Managed cloud cluster
Source code	GitHub	https://github.com/SutejNaik/CodePilot-AI

8.2 Environment Variables

The back end reads its configuration from a `.env` file placed in the backend folder (or from the hosting platform's environment settings). The names used by the code are:

Variable	Purpose
MONGO_URL	MongoDB Atlas connection string
DATABASE_NAME	Name of the database
GROQ_API_KEY	Groq API key
CODEPILOT_SECRET_KEY	Secret used to sign JWTs (the server refuses to start without it)
FRONTEND_URL	Deployed front-end origin added to the CORS allow-list
VITE_API_URL (client)	Base URL of the back end used by Axios

8.3 Running Locally

```
# Backend
cd backend
python -m venv venv
pip install -r requirements.txt
uvicorn app.main:app --reload    # http://127.0.0.1:8000

# Frontend
```

```
cd client
```

```
npm install
```

```
npm run dev # http://localhost:5173
```

Chapter 9: Limitations

- The review page currently offers three languages (Python, JavaScript and TypeScript).
- Analysis quality depends on the LLM; the output may contain mistakes or miss issues and must be verified by the developer.
- Each review is a single model call over pasted code; there is no project-wide or multi-file analysis and no static-analysis tooling.
- Very large inputs are limited by the model's context window and API rate limits.
- The AI chat conversation is not stored; only reviews are saved.
- The JWT is kept in browser local storage and there is no refresh-token mechanism; tokens expire after 7 days.
- There is no rate limiting on the review endpoints, and the free hosting tiers may be slow to start after inactivity.
- The “security issues” figure on the dashboard counts all issues found, not only those in the security category.
- The repository contains no automated test suite; testing was performed manually.

Chapter 10: Future Enhancements

- Support for more programming languages and multi-file projects.
- Combining LLM review with static-analysis tools for deterministic checks.
- Persisting AI chat conversations with each review.
- Team collaboration, shared reviews and custom project-level review rules.
- Trend analytics showing quality improvement over time.
- Email notifications and exportable reports in more formats.
- Rate limiting, refresh tokens and automated unit and integration tests.

Chapter 11: Conclusion

CodePilot AI demonstrates how a large language model can be integrated into a conventional full-stack web application to deliver a practical developer tool. The system lets users register securely, submit code in a chosen language and receive a structured review with a score, categorised issues, suggestions and a rewritten version of the code. Follow-up questions, review history, dashboard statistics and PDF reports make the tool useful beyond a single request.

The project exercised the complete development cycle: requirements, design, implementation of a REST API with authentication, database design, front-end development, integration of a third-party AI service with defensive handling of unreliable output, and cloud deployment. The limitations identified provide a clear roadmap for future work such as repository integration, multi-file analysis and automated testing.

References

- FastAPI documentation – <https://fastapi.tiangolo.com>
- React documentation – <https://react.dev>
- Vite documentation – <https://vite.dev>
- Tailwind CSS documentation – <https://tailwindcss.com>
- MongoDB Atlas and PyMongo documentation – <https://www.mongodb.com/docs>
- Groq API documentation – <https://console.groq.com/docs>
- JSON Web Token (RFC 7519) – <https://www.rfc-editor.org/rfc/rfc7519>
- passlib and bcrypt documentation – <https://passlib.readthedocs.io>
- jsPDF documentation – <https://github.com/parallax/jsPDF>
- Vercel and Render deployment documentation – <https://vercel.com/docs>,
<https://render.com/docs>

Appendix A: Screenshots

- Home / landing page

CodePilot +
— AI REVIEW

Dashboard Code Review History Profile

Log in

Analyzing repository...

AI POWERED CODE REVIEWS

Ship Better Code *Faster.*

Analyze code instantly with AI. Detect bugs, security vulnerabilities, performance issues and receive production-ready improvements.

Start Reviewing →

INTELLIGENT CODE ANALYSIS

Your code deserves a second pair of eyes.

CodePilot analyzes your code for bugs, security vulnerabilities, performance problems and bad practices — before they become expensive problems.

01

AI Code Review

Get intelligent feedback on your code with clear explanations and actionable suggestions.

02

Security Analysis

Identify potentially dangerous patterns and security weaknesses before deployment.

03

Performance

Find inefficient logic and receive suggestions to make your code cleaner and faster.

04

Actionable Reports

Understand exactly what went wrong, why it matters and how to improve it.

CODEPILOT_ENGINE

Don't just find the problem. Understand it.

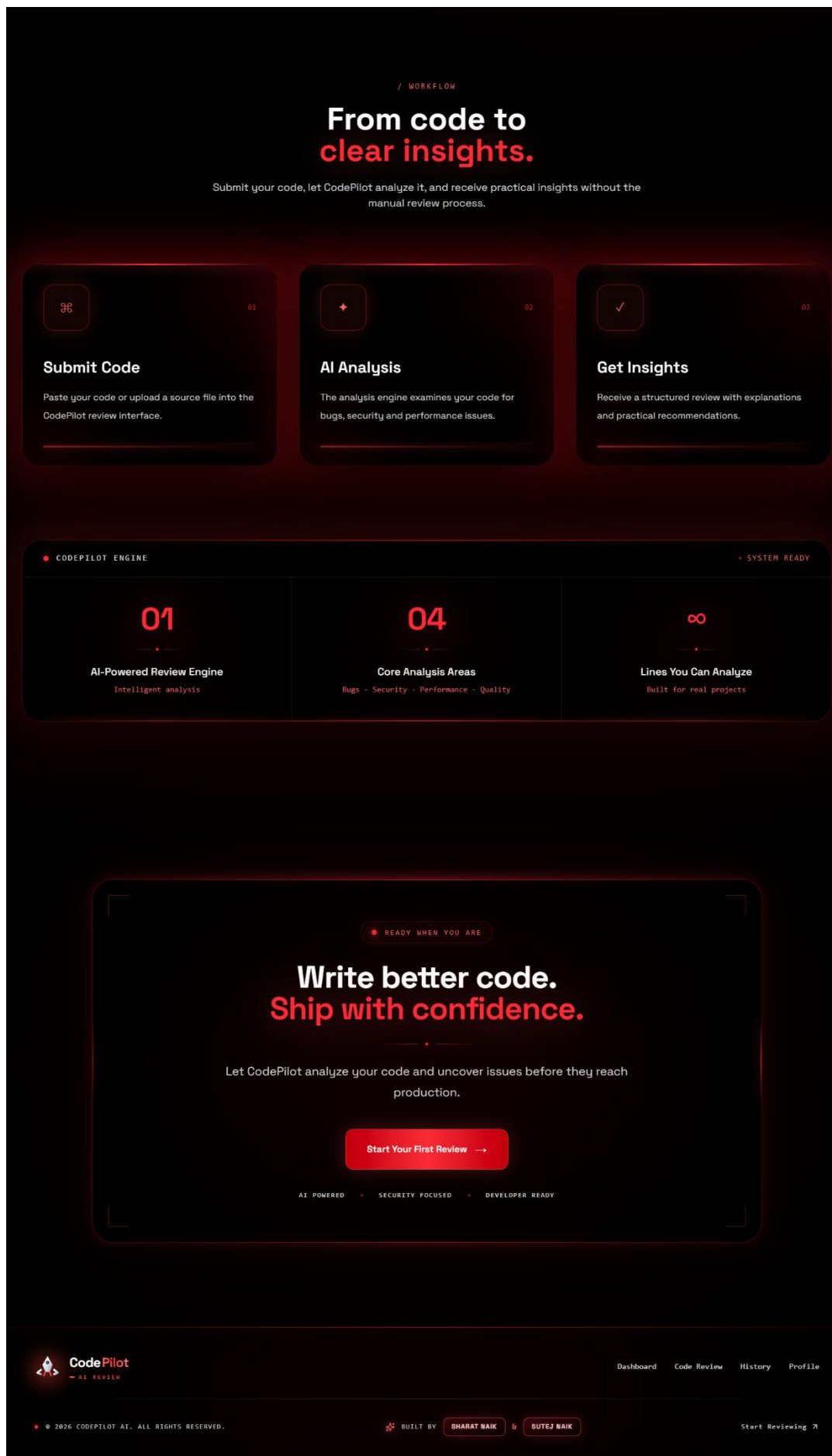
CodePilot doesn't simply throw errors at you. It explains the issue, identifies its impact and gives you a practical way to fix it.

- ✓ Detect potential bugs
- ✓ Identify security vulnerabilities
- ✓ Improve performance
- ✓ Receive AI-generated fixes

```
function calculateTotal(items) {
  let total = 0;
  items.forEach(item => {
    total += item.price;
  });
}
```

Potential issue detected

Repeated iteration may impact performance for large collections. Consider using a more efficient aggregation strategy.



- Register page



Create Account

Start analyzing your code with artificial intelligence

Full Name

Email Address

Password

Confirm Password

Create Account →

Already have an account? [Log In](#)



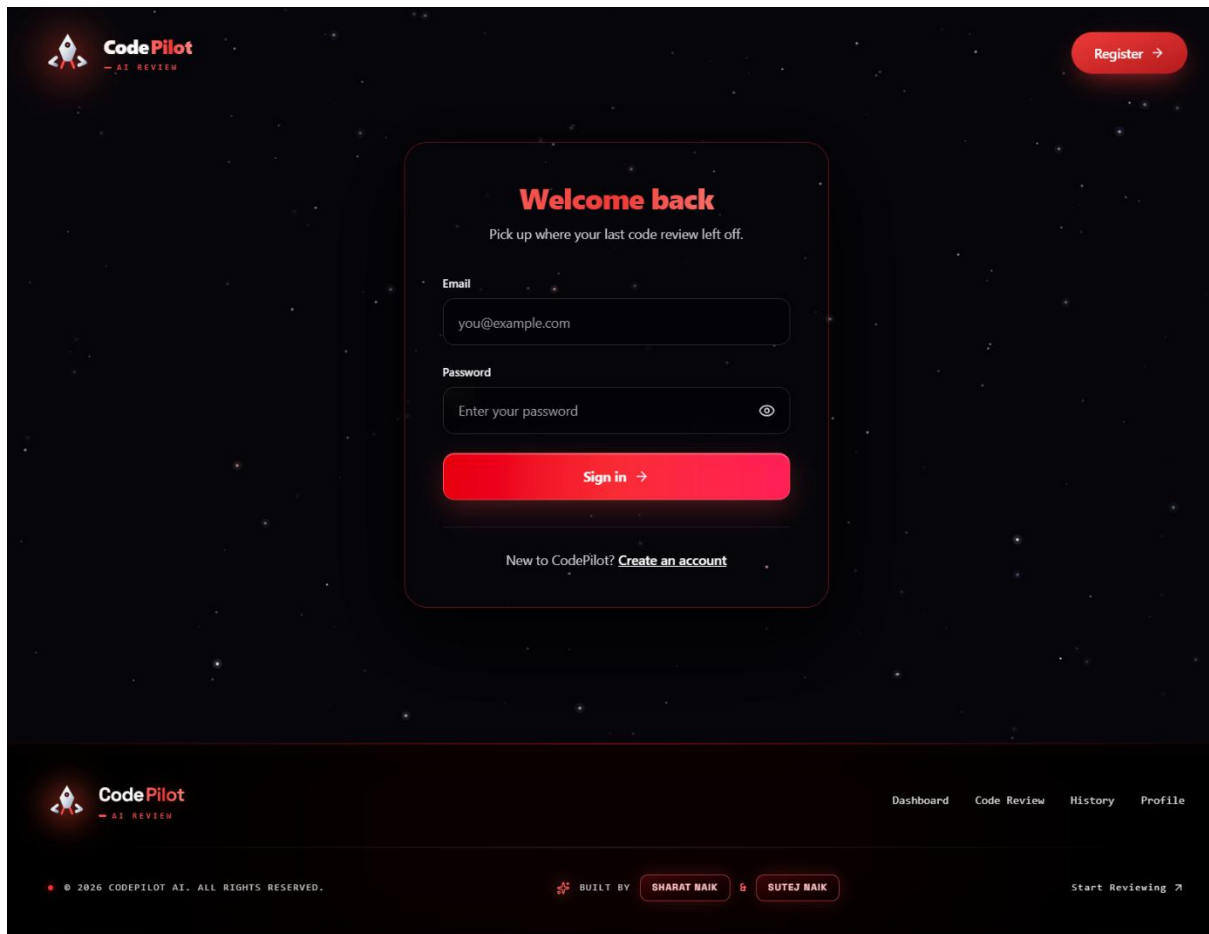
Dashboard Code Review History Profile

© 2026 CODEPILOT AI. ALL RIGHTS RESERVED.

BUILT BY SHARAT NAIK & GUTEJ NAIK

Start Reviewing ↗

- Login page



The image shows a login page for CodePilot AI Review. The background is a dark space with stars. In the top left corner is the CodePilot logo with the tagline "AI REVIEW". In the top right corner is a red "Register" button with a right arrow. The main content is a white rounded rectangle in the center. Inside, it says "Welcome back" in bold, followed by "Pick up where your last code review left off." Below this are two input fields: "Email" with the placeholder "you@example.com" and "Password" with the placeholder "Enter your password" and an eye icon for toggling visibility. A red "Sign in" button with a right arrow is below the password field. At the bottom of the white box, it says "New to CodePilot? [Create an account](#)". The footer is dark and contains the CodePilot logo on the left, navigation links "Dashboard", "Code Review", "History", and "Profile" on the right, a copyright notice "© 2026 CODEPILOT AI. ALL RIGHTS RESERVED." on the far left, a "BUILT BY" section with "SHARAT NAIK" and "SUTEJ NAIK" in the center, and a "Start Reviewing" link with an external icon on the far right.

CodePilot
— AI REVIEW

Register →

Welcome back
Pick up where your last code review left off.

Email
you@example.com

Password
Enter your password

Sign in →

New to CodePilot? [Create an account](#)

CodePilot
— AI REVIEW

Dashboard Code Review History Profile

© 2026 CODEPILOT AI. ALL RIGHTS RESERVED.

BUILT BY SHARAT NAIK SUTEJ NAIK

Start Reviewing ↗

- Dashboard



CodePilot

AI REVIEW

DashboardCode ReviewHistoryProfile

Log out

CODEPILOT DASHBOARD

Your coding overview.

Track your code reviews, quality scores and security findings from one place.

Review New Code

OVERVIEW

Review statistics

Total Reviews

3

Code reviews completed

Average Score

65%

Overall code quality

Security Issues

8

Potential issues detected

Languages

2

Languages reviewed

ACTIVITY

Recent reviews

View history

Recent Reviews

View All

PYTHON	10/5/2026, 2:56:56 PM	78%	2 Issue(s)
PYTHON	10/5/2026, 2:55:12 PM	48%	2 Issue(s)
TYPESCRIPT	10/7/2026, 4:03:47 PM	68%	4 Issue(s)

SHORTCUTS

Quick actions

Quick Actions

</>

Review Code

Start a new AI code review

🕒

View History

Check previous reviews

👤

Profile

Manage your account

ANALYTICS

Review insights

Understand how you use CodePilot over time.

📊 Code Quality

AI analysis coverage

- Security Issues
- Bugs
- Performance Problems
- Best Practices
- Maintainability
- Readability

</> Supported Languages

Available for analysis

Python

Fully Supported

JavaScript

Supported

Java

Supported

C++

Supported

Ready to review more code?

Run another AI-powered code analysis.

Start Review



CodePilot

AI REVIEW

DashboardCode ReviewHistoryProfile

© 2026 CODEPILOT AI. ALL RIGHTS RESERVED.

BUILT BYSHARAT NAIKSUTEJ NAIK

Start Reviewing

- Review page

The screenshot displays the CodePilot AI Review interface. At the top, the navigation bar includes 'Dashboard', 'Code Review' (active), 'History', and 'Profile', along with a 'Log out' button. The main header features the CodePilot logo and the tagline 'Write better code. Ship with confidence.' Below this, a sub-header states: 'CodePilot analyzes your code for bugs, security vulnerabilities, performance issues and code quality.'

The interface is divided into two main sections: 'Code Review Workspace' and 'Review Results'.

Code Review Workspace: This section allows users to analyze their source code. It includes a 'Source Code' editor with a Python example function: `def get_user_data(user_id): query = "SELECT * FROM users WHERE id = '" + user_id + '""; return db.execute(query)`. A 'Review Code' button is visible at the bottom right of this section.

Review Results: This section provides an 'AI-powered analysis of your source code'. It includes a 'Reviewed Code' tab showing the original source code and a 'Review Summary' section. The summary shows an overall code quality score of 45, categorized as 'Attention required'. It also lists specific findings: 3 issues, 1 security vulnerability, 0 performance issues, and 0 quality issues. An 'AI Summary' section provides a detailed analysis of the code, highlighting an SQL injection risk and the need for parameterized queries and input validation.

At the bottom of the interface, there are four summary cards: 'BUGS' (1), 'SECURITY' (1), 'PERFORMANCE' (0), and 'QUALITY' (0).

Issues Found

PROBLEMS DETECTED BY AI

3 detected

01

SQL Injection vulnerability

SECURITYHIGH

User input is concatenated directly into SQL query, allowing injection attacks.

</> Line 4

02

Undefined variable 'db'

BUGMEDIUM

The function references 'db' which is not defined in the scope, causing NameError at runtime.

</> Line 5

03

Missing type hints and input validation

BAD PRACTICELOW

Function lacks type annotations and does not validate that user_id is of expected type.

</> Line 3

AI Suggestions

RECOMMENDED IMPROVEMENTS

1

Use parameterized queries to prevent SQL injection.

2

Define or pass a database connection object instead of relying on an undefined global.

3

Add type hints and validate input types.

4

Handle database errors with try/except and ensure connections are closed.

Updated Code

AI SUGGESTED IMPROVED VERSION

IMPROVED

Improved Code

AI generated improved version

Copy

```

import sqlite3
from typing import Any, List, Tuple

def get_connection() -> sqlite3.Connection:
    return sqlite3.connect('example.db')

def get_user_data(user_id: int) -> List[Tuple[Any, ...]]:
    """Fetch user data for the given user_id using a parameterized query."""
    if not isinstance(user_id, int):
        raise TypeError("user_id must be an integer")
    query = 'SELECT * FROM users WHERE id = ?'
    conn = get_connection()
    try:
        with conn:
            cur = conn.execute(query, (user_id,))
            return cur.fetchall()
    finally:
        conn.close()

```

AI Assistant

ASK ANYTHING ABOUT YOUR CODE

CodePilot AI

Ask AI About Your Code

ASK SOMETHING TO UNDERSTAND ABOUT YOUR REVIEW

AI ASSISTANT

Why is this insecure?

The original version builds the SQL statement by concatenating the raw 'user_id' value into the query string:

```

"""python
query = "SELECT * FROM users WHERE id = " + user_id + ""
...
"""

```

If 'user_id' comes from an external source (e.g., a web request, form input, or API call), an attacker can inject arbitrary SQL code by supplying a specially crafted value, such as:

```

...
user_id = "1; DROP TABLE users; --"
...

```

Ask anything about your review...

HOW TO USE AI REVIEW: Click on the icon

CODEPILOT AI ENGINE

COMPLETE

WHAT CODEPILOT CHECKS

One review. Four layers of intelligence.

Go beyond basic linting with an AI-powered review focused on the things that actually matter.

ANALYSIS LAYER

Bug Detection

Identify logical errors, runtime problems and unreliable code paths.

EXPLORE ANALYSIS >

ANALYSIS LAYER

Security Analysis

Detect vulnerable patterns and potential security weaknesses.

EXPLORE ANALYSIS >

ANALYSIS LAYER

Performance

Find inefficient operations, bottlenecks and unnecessary work.

EXPLORE ANALYSIS >

ANALYSIS LAYER

Code Quality

Improve readability, maintainability and engineering practices.

EXPLORE ANALYSIS >

KEEP BUILDING

Ready for another review?

Paste another snippet and let CodePilot find what you missed.

Try another snippet >

CodePilot

AI REVIEW

Dashboard Code Review History Profile

© 2024 CODEPILOT AI. ALL RIGHTS RESERVED.

BUILT BY SHARAT NAIK SUTEJ NAIK

Start Reviewing >

34

- History page



CodePilot

— AI REVIEW

[Dashboard](#)[Code Review](#)[History](#)[Profile](#)

[Log out](#)

REVIEW ARCHIVE

Review History

Browse, search and revisit your previous AI-powered code reviews.

Refresh

TOTAL REVIEWS3

LANGUAGES USED2

CURRENTLY SHOWING3

Search reviews, languages or code...

All Languages

Newest First

Your Reviews

3 REVIEWS FOUND

AI ANALYZED



PYTHON

REVIEWED

10/5/2026, 2:56:58 PM

CODE QUALITY

78%

ISSUES FOUND

2

AI ANALYSIS

COMPLETE

VIEW REVIEW

CODEPILOT AI

REVIEW RICHMILK



PYTHON

REVIEWED

10/5/2026, 2:55:12 PM

CODE QUALITY

48%

ISSUES FOUND

2

AI ANALYSIS

COMPLETE

VIEW REVIEW

CODEPILOT AI

REVIEW RICHMILK



TYPESCRIPT

REVIEWED

10/1/2026, 4:03:47 PM

CODE QUALITY

68%

ISSUES FOUND

4

AI ANALYSIS

COMPLETE

VIEW REVIEW

CODEPILOT AI

REVIEW RICHMILK



CodePilot

— AI REVIEW

[Dashboard](#)[Code Review](#)[History](#)[Profile](#)

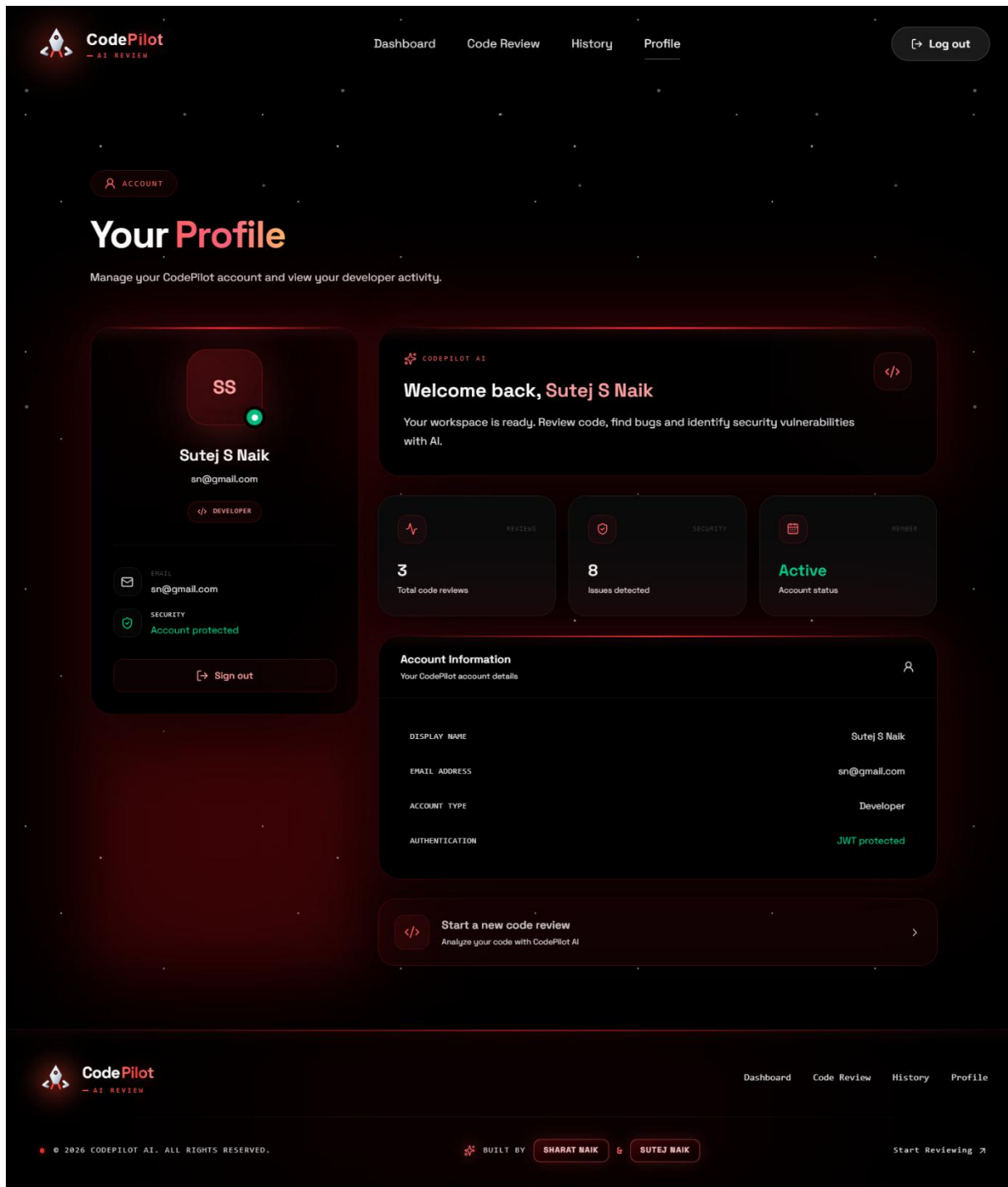
© 2026 CODEPILOT AI. ALL RIGHTS RESERVED.

BUILT BYSHARAT NAIKSUTEJ NAIK

Start Reviewing

35

- Profile page





Submitted by :

SUTEJ NAIK

 Web App: <https://codepilot-ai-nine.vercel.app/>

 GitHub: <https://github.com/SutejNaik/CodePilot-AI.git>